

PowerShell Security & System Administration Activity

Objective

Learn how to properly interact with the system file structure using formal PowerShell cmdlets. This exercise focuses on navigating directories safely, establishing sandboxed workspaces, performing logs updates, and maintaining rigid scripting baselines essential for defensive security workflows.

DEFENSIVE ENGINEERING STANDARDS NOTE: While PowerShell supports legacy compatibility shortcuts like 'cd', 'ls', 'dir', and 'rm', these aliases are strictly prohibited in enterprise incident response playbooks and defense scripts. Utilizing full, formal cmdlets (e.g., Set-Location, Get-ChildItem) ensures predictable execution, prevents alias collision errors on non-Windows endpoints, and guarantees strict cross-platform compatibility across heterogeneous enterprise networks (Windows, Linux, and macOS running PowerShell Core).

Understanding the Tilde (~) Shortcut

The tilde symbol (~) is an interactive shortcut in PowerShell that stands for your active User Profile Home directory. Instead of hardcoding or typing a long system account path, you can use ~ to instantly reference it.

Example:

*If your security profile username is "John", typing ~ points directly to: C:\Users\John\
Therefore, writing ~\Downloads tells PowerShell to go to: C:\Users\John\Downloads*

** Professional Tip: While ~ works perfectly inside native PowerShell navigation, external command-line security tools, legacy batch engines, or automated scripts might not parse it correctly. For maximum compatibility across all heterogeneous programs, tools, and platforms, security professionals prioritize using the global variable \$HOME instead.*

Step-by-Step Activity

Step 1: Find Your Current Location

Before executing any directory actions, check where your terminal environment is currently pointing.

```
Get-Location
# Displays the precise file path of your current directory
# Shortcut alternative: pwd
```

Step 2: Navigate to the Downloads Folder

Change your active directory path over to your user profile's Downloads folder using the clear user profile tilde path (~).

```
Set-Location -Path ~\Downloads
# Navigates seamlessly directly into the standard Downloads folder
# Shortcut alternative: cd ~\Downloads
# Advanced equivalent: Set-Location -Path "$HOME\Downloads"
```

Step 3: List Folder Contents

See what items and documents already occupy your general Downloads folder context.

```
Get-ChildItem
# Examines the workspace to reveal files and folders inside
# Shortcut alternatives: ls OR dir
```

Step 4: Create a New Folder (Sandbox)

Form a new dedicated directory container named test_for_text to serve as your clean, isolated sandbox area. This ensures test commands do not impact personal or corporate assets.

```
New-Item -ItemType Directory -Name "test_for_text"
# Creates an isolated folder sandbox without touching external files
# Shortcut alternative: mkdir test_for_text
```

Step 5: Verify Folder Creation

Perform a folder listing check to verify that your new test_for_text directory sandbox was successfully provisioned.

```
Get-ChildItem
# Confirms the existence of the new folder dynamically from the shell
# Shortcut alternatives: ls OR dir
```

Step 6: Navigate into the New Folder

Pivot into your newly generated workspace to begin local file operations within your isolated perimeter.

```
Set-Location -Path "test_for_text"
# Sets the terminal environment inside the new sandbox folder
# Shortcut alternative: cd test_for_text
```

Step 7: Create a New Text File

Generate a clear, standalone file named test.txt containing the exact base string value "testt".

```
Set-Content -Path "test.txt" -Value "testt"
# Builds a brand new file and populates it with the initial word
# Shortcut alternative: sc test.txt -Value "testt"
```

Step 8: Read the File Contents

Confirm that the document string was generated cleanly by reading the text directly from the host terminal window.

```
Get-Content -Path "test.txt"
# Outputs the internal text of the file to verify accuracy
# Shortcut alternative: cat test.txt
```

Step 9: Update the File (Append Text)

Perform an operational file modification step by adding an additional row of string content without clearing out your old data line.

```
Add-Content -Path "test.txt" -Value "This is an updated line."  
# Appends the new sentence cleanly to the very bottom of the document  
# (Note: This creates a new line because Set-Content automatically added a  
line break in Step 7. Add-Content's shortcut alternative is ac)
```

Step 10: Read the Updated File

Read the document text block a second time to ensure the structural updates merged perfectly.

```
Get-Content -Path "test.txt"  
# Confirms that both lines of data now correctly reside in the file  
# Shortcut alternative: cat test.txt
```

Step 11: View File Properties

Examine basic, standard metadata elements bound to the file object context, such as size parameters and modification logs.

```
Get-Item -Path "test.txt"  
# Formats basic attributes like exact size, write timestamp, and path flags  
# Shortcut alternative: gi test.txt
```

Step 12: List Directory Contents Again

Run a local directory query to observe the file component existing inside the folder.

```
Get-ChildItem  
# Displays the isolated file context safely within your sandbox environment  
# Shortcut alternatives: ls OR dir
```

Step 13: Delete the File

Purge the test content item directly from your sandbox folder workspace.

```
Remove-Item -Path "test.txt"  
# Completely deletes the targeted text file  
# Shortcut alternative: rm test.txt
```

Step 14: Final Verification List

Execute a terminal verification query to confidently prove that the text file was completely wiped out from the context folder.

```
Get-ChildItem  
# Confirms the directory structure is empty and file deletion succeeded  
# Shortcut alternatives: ls OR dir
```